ASSIGNMENT 5 – CO5

DATA PERSISTENCE ENGINEERING – FILES AND DATABASES

1. INTRODUCTION

Data persistence is an important concept in software development. It refers to the process of storing data permanently so that it remains available even after a program has been closed or the system has been restarted.

Java provides several methods for storing and managing data, including text files, binary files, buffered streams, object serialization and relational databases using JDBC.

This assignment focuses on understanding how Java applications store data safely, efficiently and correctly. It explores file handling, database connectivity, transaction management, SQL injection prevention and performance measurement.

The assignment uses a university transport office bus-pass management system as the main example. The system stores student bus-pass information and supports operations such as adding new records, searching for a pass and listing passes based on route numbers.

2. OBJECTIVES

- To understand Java file handling using byte streams and character streams.
- To design suitable text and binary file formats.
- To understand buffered input and output operations.
- To analyse the performance of buffered and unbuffered file reading.
- To understand the Data Access Object (DAO) design pattern.
- To study object serialization and the use of the transient keyword.
- To understand JDBC connectivity and database operations.
- To implement safe database transactions using commit and rollback.
- To identify SQL injection vulnerabilities and prevent them using PreparedStatement.
- To compare Statement, PreparedStatement and batch processing.
- To evaluate suitable data storage methods for large-scale applications.

3. PART A – PERSISTENCE DESIGN AND PSEUDOCODE

TASK A1 – DESIGN A FILE FORMAT

3.1 Introduction

The university transport office maintains approximately 8,000 bus-pass records every year. Each record contains the pass ID, student register number, student name, route number, boarding stop, validity date and amount paid.

A suitable file format is required to store and retrieve these records efficiently.

3.2 A1.1 – Record Representation

A. Delimited Text File Format

A delimited text file stores each bus-pass record as a line containing fields separated by a delimiter such as a comma.

Example:

P1001,192311001,Arun,12,Main Gate,2027-03-31,4500

Field order:

1. Pass ID
2. Student Register Number
3. Student Name
4. Route Number
5. Boarding Stop
6. Validity Date
7. Amount Paid

If a student name or boarding stop contains a comma, the field should be enclosed in quotation marks or an appropriate escaping method should be used. This prevents the delimiter from being mistaken for a field separator.

B. Binary File Format

A binary file stores data in a structured format using DataOutputStream.

The fields can be written in the following order:

- Pass ID – UTF string
- Register Number – UTF string

- Student Name – UTF string
- Route Number – Integer
- Boarding Stop – UTF string
- Validity Date – UTF string
- Amount Paid – Double

Binary storage provides structured data representation and avoids delimiter-related problems.

3.3 A1.2 – Design of File Operations

The transport office requires three main operations.

1. Add a New Pass

1. Accept the bus-pass details from the user.
2. Validate the entered information.
3. Open the file using FileWriter and BufferedWriter.
4. Append the new record to the file.
5. Close the file after writing.

2. Find a Pass by Pass ID

1. Accept the pass ID.
2. Open the file using FileReader and BufferedReader.
3. Read each line from the file.
4. Split the line into individual fields.
5. Check whether the pass ID matches.
6. Display the matching record.
7. Display a suitable message if the record is not found.

3. List Passes by Route

1. Accept the route number.
2. Read the file line by line.
3. Separate the fields in each record.
4. Compare the route number.
5. Display all matching records.

BufferedReader and BufferedWriter reduce unnecessary input and output operations by processing data through buffers.

3.4 A1.3 – Design for Damaged Data

A file may contain damaged or incomplete records due to unexpected system shutdowns, power failures or incorrect data entry.

The following methods can help handle these situations:

- **Damaged line:** Check the number of fields in every record. Skip invalid records and report them.
- **Missing file:** Check whether the file exists before reading. If it does not exist, create it when appropriate or display a clear error message.
- **Incomplete write:** Use temporary files and atomic replacement where supported. Validate the completed records before treating the file as updated.

3.5 Alternative Concept – RandomAccessFile

RandomAccessFile can be used when records have fixed lengths. Each record can be accessed by calculating its position in the file.

The record position is calculated as:

Record Position = Record Index × Record Size

A sequential text-file search requires checking records one by one, giving O(n) time complexity. A fixed-length record can be accessed directly using its calculated position in O(1) time when its record index is known.

However, fixed-length fields may waste storage space because shorter values must be padded to the specified field size.

TASK A2 – DON'T READ WHAT YOU DON'T NEED

3.6 Introduction

Reading large files one byte at a time can reduce application performance. This task considers a 50 MB attendance log file that is read using FileInputStream.read() to count the number of lines.

3.7 A2.1 – Predict the Cost

A 50 MB file contains approximately 52,428,800 bytes when 1 MB is treated as 1,048,576 bytes.

When the program reads one byte at a time, it performs approximately 52 million read operations.

This approach is inefficient because repeated read calls create additional method-call and input/output overhead.

3.8 A2.2 – Redesign Using Buffering

A buffered stream can improve performance by reading a block of bytes into memory instead of requesting each byte separately.

The redesigned approach uses BufferedInputStream or a byte array buffer.

Advantages include:

- Reduces repeated input/output operations.
- Improves file-reading performance.
- Processes multiple bytes in a single operation.
- Makes large-file processing more efficient.

3.9 A2.3 – When Buffering Does Not Help

Buffering may provide little benefit when the application reads very small files or when the data is already available in memory.

It may also provide limited improvement when the main performance bottleneck is elsewhere, such as slow processing or network communication.

3.10 Alternative Concept – Files.readAllLines()

Files.readAllLines() reads the complete file into memory as a list of strings.

Its time complexity is O(n), where n represents the amount of file data. Its memory requirement is also O(n), because the complete file contents are stored in memory.

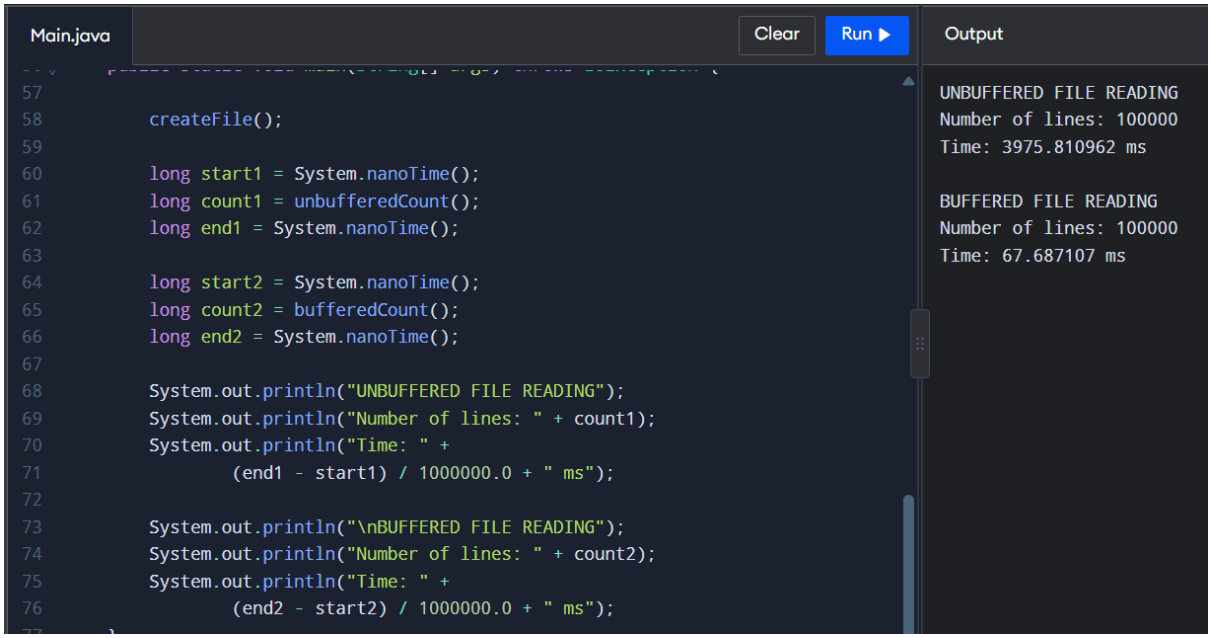
Buffered reading also requires O(n) total processing time, but it can use O(b) additional memory, where b is the buffer size.

Therefore, buffered reading is more suitable for large files, while readAllLines() is convenient for smaller files that comfortably fit in memory.

PROGRAM:

```
import java.io.*;
public class Main {
   static String fileName = "attendance.txt";
   // Create sample attendance file
   static void createFile() throws IOException {
      try (BufferedWriter writer =
               new BufferedWriter(new FileWriter(fileName))) {
```

```
            for (int i = 1; i <= 100000; i++) {
                writer.write("Student " + i + " Present");
                writer.newLine();
            }
        }
    }
    // Unbuffered reading
    static long unbufferedCount() throws IOException {
        long lines = 0;
        try (FileInputStream input =
                new FileInputStream(fileName)) {
            int data;
            while ((data = input.read()) != -1) {
                if (data == '\n') {
                    lines++;
                }
            }
        }
        return lines;
    }
    // Buffered reading
    static long bufferedCount() throws IOException {
        long lines = 0;
        try (BufferedReader reader =
                new BufferedReader(new FileReader(fileName))) {
            while (reader.readLine() != null) {
                lines++;
            }
        }
        return lines;
    }
    public static void main(String[] args) throws IOException {
        createFile();
        long start1 = System.nanoTime();
        long count1 = unbufferedCount();
        long end1 = System.nanoTime();
        long start2 = System.nanoTime();
        long count2 = bufferedCount();
        long end2 = System.nanoTime();
        System.out.println("UNBUFFERED FILE READING");
        System.out.println("Number of lines: " + count1);
        System.out.println("Time: " +
            (end1 - start1) / 1000000.0 + " ms");
        System.out.println("\nBUFFERED FILE READING");
        System.out.println("Number of lines: " + count2);
        System.out.println("Time: " +
            (end2 - start2) / 1000000.0 + " ms");
    }
}
```



TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN

3.11 Introduction

In a student management project, StudentDB, CourseDB and MarksDB perform similar database operations. Each class opens a connection, executes SQL queries, processes the ResultSet and closes the connection.

Repeating the same database logic in multiple classes makes the application difficult to maintain.

3.12 A3.1 – Identify the Common Pattern

The repeated operations are:

1. Establishing a database connection.
2. Preparing and executing SQL statements.
3. Reading the ResultSet.
4. Converting database records into Java objects.
5. Closing database resources.

The parts that differ are the table names, column names, object types and SQL operations.

3.13 A3.2 – Generic Data-Access Interface

The Data Access Object (DAO) pattern separates database operations from application logic.

A generic interface named Dao<T, K> can define the following methods:

- insert(T object)
- findById(K id)
- findAll()
- update(T object)
- delete(K id)

StudentDao implements this interface and handles operations related to student records.

This approach reduces code duplication and improves maintainability.

3.14 A3.3 – Mapping Objects to Tables

The Student class can be mapped to a database table named student.

Java Field	Database Column	Description
studentId	student_id	Primary key
name	name	Student name
email	email	Email address
department	department	Department name

The student_id column uniquely identifies each student record.

3.15 Alternative Concept – SQL Inside Service or GUI Classes

If SQL queries are written directly inside service or GUI classes, database logic becomes distributed throughout the application.

When a table column is renamed, multiple classes may need modifications.

Using a DAO design centralises database operations, so changes can usually be handled in fewer places.

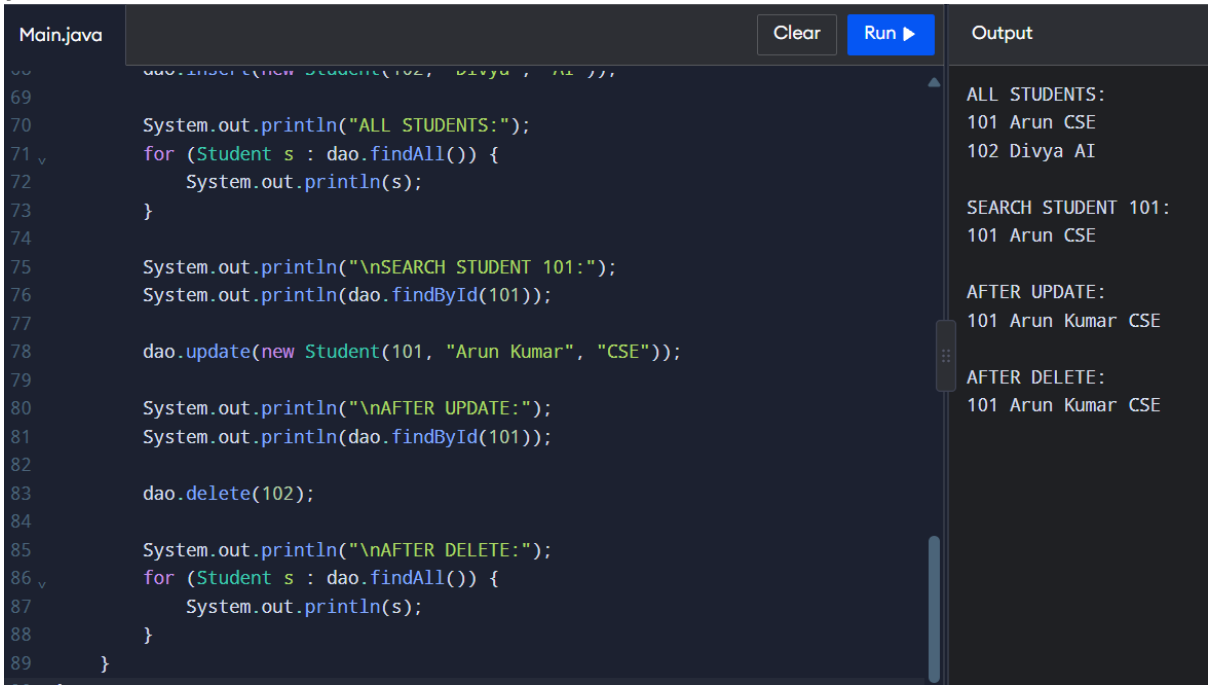
Both designs have broadly similar database operation time complexity when they execute equivalent SQL queries. The main advantage of DAO is improved maintainability and separation of responsibilities.

PROGRAM:

```
import java.util.*;
class Student {
   int id;
   String name;
   String department;
```

```
    Student(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
    public String toString() {
        return id + " " + name + " " + department;
    }
}
interface Dao<T, K> {
    void insert(T object);
    T findById(K id);
    List<T> findAll();
    void update(T object);
    void delete(K id);
}
class StudentDao implements Dao<Student, Integer> {
   private List<Student> students = new ArrayList<>();
   public void insert(Student student) {
        students.add(student);
    }
    public Student findById(Integer id) {
        for (Student s : students) {
            if (s.id == id) {
                return s;
            }
        }
        return null;
    }
    public List<Student> findAll() {
        return students;
    }
    public void update(Student student) {
        for (int i = 0; i < students.size(); i++) {
            if (students.get(i).id == student.id) {
                students.set(i, student);
                return;
            }
        }
    }
    public void delete(Integer id) {
        students.removeIf(s -> s.id == id);
    }
}
public class Main {
    public static void main(String[] args) {
        StudentDao dao = new StudentDao();
        dao.insert(new Student(101, "Arun", "CSE"));
        dao.insert(new Student(102, "Divya", "AI"));
 System.out.println("ALL STUDENTS:");
```

```
        for (Student s : dao.findAll()) {
            System.out.println(s);
        }
        System.out.println("\nSEARCH STUDENT 101:");
        System.out.println(dao.findById(101));
        dao.update(new Student(101, "Arun Kumar", "CSE"));
        System.out.println("\nAFTER UPDATE:");
        System.out.println(dao.findById(101));
        dao.delete(102);
        System.out.println("\nAFTER DELETE:");
        for (Student s : dao.findAll()) {
            System.out.println(s);
        }
    }
}
```



TASK A4 – WHAT SHOULD BE SERIALIZED?

3.16 Introduction

Serialization is the process of converting a Java object into a byte stream so that it can be stored in a file or transferred.

A hostel management system contains Hostel, Room and Resident objects. These objects form an object graph because one object contains references to other objects.

3.17 A4.1 – Decide What to Persist

The Hostel object contains a list of Room objects, and each Room contains a list of Resident objects.

The following information can be stored:

- Hostel details
- Room details
- Resident names
- Resident register numbers

The password and live database Connection should not be serialized.

The database Connection must be marked transient because it represents a live resource that cannot be restored meaningfully from a saved object.

Sensitive password information should not be stored as ordinary serialized data. It should be handled through secure authentication storage.

When the Hostel object is serialized, its reachable serializable objects are also serialized, subject to Java serialization rules. The transient fields are not included in the serialized data.

3.18 A4.2 – Plan for Change

When a new phone number field is added to the Resident class, older serialized files may still be read if serialization compatibility is maintained.

The serialVersionUID identifies the version of a serializable class.

If the serialVersionUID remains compatible, a missing field in an older file is generally assigned its default value when deserialized. If incompatible class versions have different serialVersionUID values, an InvalidClassException may occur.

3.19 Alternative Concept – CSV or JSON

CSV and JSON are text-based formats that can store hostel information.

CSV advantages:

- Simple format.
- Easy to inspect and edit.
- Can be opened using spreadsheet applications.

JSON advantages:

- Supports structured and nested data.
- More flexible for object-based information.
- Easier to extend with additional fields.

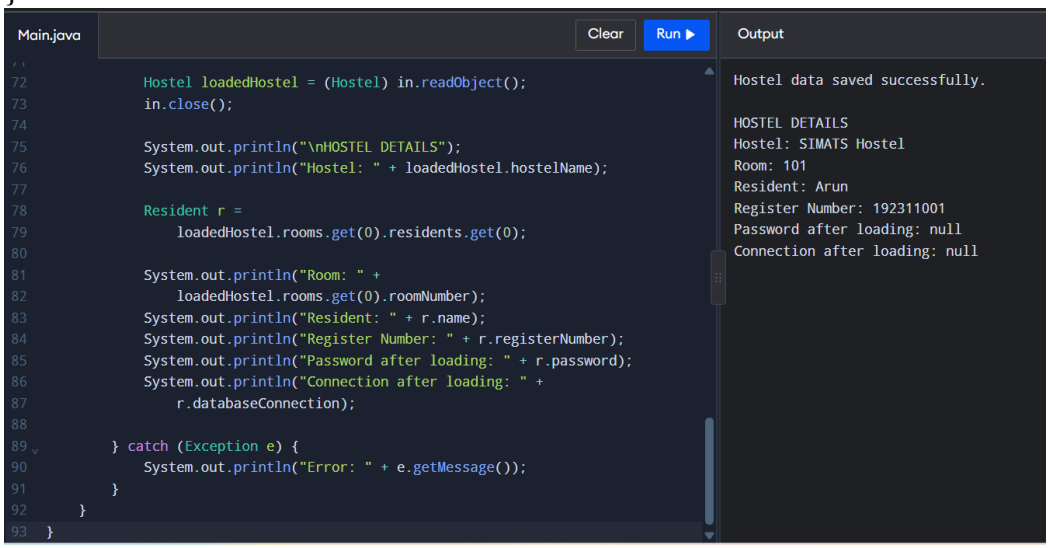
Java serialization is convenient for saving Java object graphs, while CSV and JSON provide better readability and interoperability.

The total save and load time generally grows with the number of residents, represented as O(n). Actual performance and file size depend on the stored fields and format.

PROGRAM:

```
import java.io.*;
import java.util.*;
class Resident implements Serializable {
   private static final long serialVersionUID = 1L;
   String name;
   String registerNumber;
   transient String password;
   transient Object databaseConnection;
   Resident(String name, String registerNumber, String password) {
      this.name = name;
      this.registerNumber = registerNumber;
      this.password = password;
      this.databaseConnection = new Object();
   }
}
class Room implements Serializable {
   private static final long serialVersionUID = 1L;
   int roomNumber;
   ArrayList<Resident> residents = new ArrayList<>();
   Room(int roomNumber) {
      this.roomNumber = roomNumber;
   }
}
class Hostel implements Serializable {
   private static final long serialVersionUID = 1L;
   String hostelName;
   ArrayList<Room> rooms = new ArrayList<>();
   Hostel(String hostelName) {
      this.hostelName = hostelName;
   }
}
public class Main {
   public static void main(String[] args) {
      try {
         Hostel hostel = new Hostel("SIMATS Hostel");
```

```
            Room room = new Room(101);
            room.residents.add(
                new Resident("Arun", "192311001", "secret123")
            );
            hostel.rooms.add(room);
            // Serialize object
            ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("hostel.dat")
            );
            out.writeObject(hostel);
            out.close();
            System.out.println("Hostel data saved successfully.");
            // Deserialize object
            ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("hostel.dat")
            );
            Hostel loadedHostel = (Hostel) in.readObject();
            in.close();
            System.out.println("\nHOSTEL DETAILS");
            System.out.println("Hostel: " + loadedHostel.hostelName);
            Resident r =
                loadedHostel.rooms.get(0).residents.get(0);
            System.out.println("Room: " +
                loadedHostel.rooms.get(0).roomNumber);
            System.out.println("Resident: " + r.name);
            System.out.println("Register Number: " + r.registerNumber);
            System.out.println("Password after loading: " + r.password);
            System.out.println("Connection after loading: " +
                r.databaseConnection);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```



TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY

3.20 Introduction

Database applications must maintain data correctness and protect user information. Transactions and prepared statements help achieve these goals.

3.21 A5.1 – Transaction Design

Consider a student transferring ₹500 from a hostel wallet to a canteen wallet.

Both wallet updates must succeed together. If one update fails, the other update must also be cancelled.

The transaction procedure is:

1. Establish a JDBC connection.
2. Disable auto-commit.
3. Deduct ₹500 from the hostel wallet.
4. Add ₹500 to the canteen wallet.
5. Commit the transaction if both updates succeed.
6. Roll back the transaction if any update fails.
7. Restore the original auto-commit setting.

This prevents money from being lost or created because of a partial transfer.

3.22 A5.2 – SQL Injection Analysis

SQL injection is a security vulnerability that occurs when user input is directly combined with an SQL query.

For example, building a login query through string concatenation allows specially crafted input to change the meaning of the SQL statement.

This may allow an attacker to bypass authentication.

Safe approach using PreparedStatement:

PreparedStatement uses placeholders for user inputs. The values are supplied separately from the SQL command.

Advantages:

- Prevents user input from changing the SQL query structure.
- Reduces SQL injection risks.

- Supports parameterised database operations.
- Improves query organisation and reuse.

3.23 Alternative Concept – Separate Auto-Commit Updates

If two wallet updates are executed as independent auto-commit statements, the first update may become permanent before the second update fails.

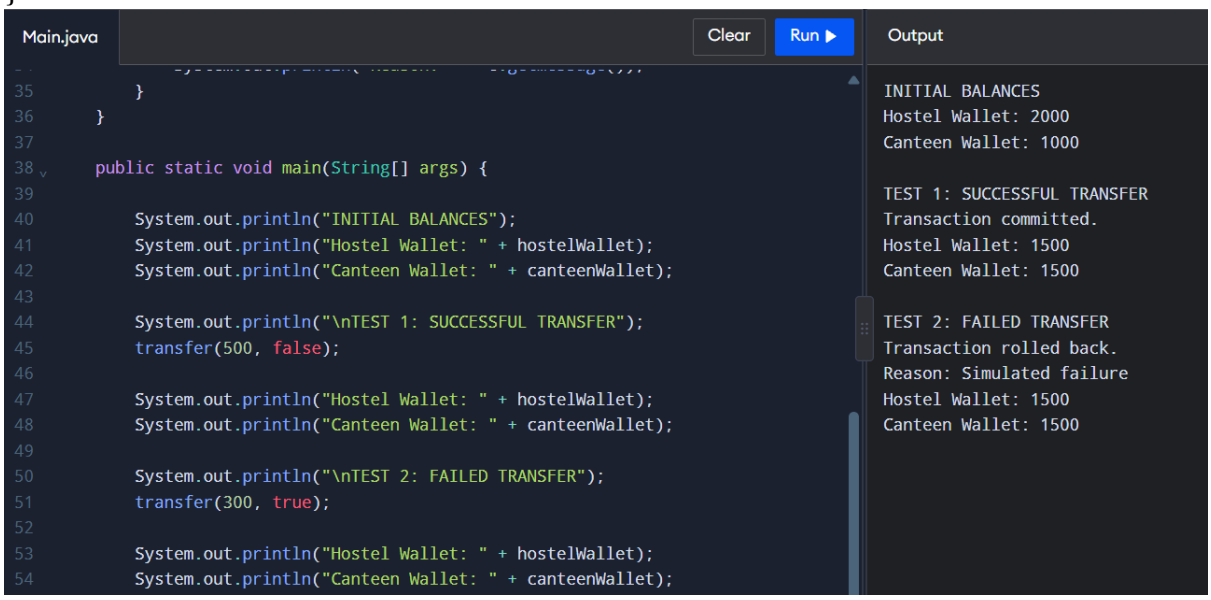
This can result in an incorrect balance.

A single transaction provides atomicity, ensuring that both updates are completed or neither is completed.

PROGRAM:

```
public class Main {
   static int hostelWallet = 2000;
   static int canteenWallet = 1000;
   static void transfer(int amount, boolean simulateFailure) {
      // Save original balances
      int oldHostel = hostelWallet;
      int oldCanteen = canteenWallet;
      try {
         if (amount <= 0 || amount > hostelWallet) {
            throw new Exception("Invalid transfer amount");
         }
         hostelWallet = hostelWallet - amount;
         if (simulateFailure) {
            throw new Exception("Simulated failure");
         }
         canteenWallet = canteenWallet + amount;
         System.out.println("Transaction committed.");
      } catch (Exception e) {
         // Rollback
         hostelWallet = oldHostel;
         canteenWallet = oldCanteen;
         System.out.println("Transaction rolled back.");
         System.out.println("Reason: " + e.getMessage());
      }
   }
   public static void main(String[] args) {
      System.out.println("INITIAL BALANCES");
      System.out.println("Hostel Wallet: " + hostelWallet);
      System.out.println("Canteen Wallet: " + canteenWallet);
      System.out.println("\nTEST 1: SUCCESSFUL TRANSFER");
      transfer(500, false);
      System.out.println("Hostel Wallet: " + hostelWallet);
      System.out.println("Canteen Wallet: " + canteenWallet);
      System.out.println("\nTEST 2: FAILED TRANSFER");
      transfer(300, true);
```

```
        System.out.println("Hostel Wallet: " + hostelWallet);
        System.out.println("Canteen Wallet: " + canteenWallet);
    }
}
```



4. PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE

4.1 Introduction

This task implements the bus-pass file store designed in Task A1 and the buffered line counter designed in Task A2.

The implementation must support adding records, searching for a pass and handling damaged records.

4.2 B1.1 – Validate the Implementation

The following test cases are required:

Test Case	Input or Condition	Expected Result
Normal case	Existing pass ID P1003	Display the matching record
Missing record	Pass ID P9999	Return null or show not found
Damaged record	Record with incorrect field count	Skip the damaged record

These tests verify whether the file store can handle valid, missing and damaged records correctly.

4.3 B1.2 – Compare with the Design

The implementation follows the pseudocode by reading the file line by line using BufferedReader.

Each line is separated into fields and checked before searching for the requested pass ID.

Damaged lines are skipped so that they do not interrupt the complete search process.

4.4 B1.3 – Measure the Effect of Buffering

The performance of unbuffered and buffered file reading must be measured using files of different sizes.

File Size	Unbuffered Reading Time	Buffered Reading Time
1 MB	To be measured	To be measured
10 MB	To be measured	To be measured
50 MB	To be measured	To be measured

The results should be recorded after executing both implementations.

Buffered reading is expected to reduce repeated input/output overhead, although the actual improvement depends on the computer, storage device and Java runtime.

4.5 B1.4 – Measure Growth

The search performance must be measured using different numbers of bus-pass records.

Number of Records	Search Time
1,000	To be measured
10,000	To be measured
100,000	To be measured

A sequential search checks records one by one. Therefore, its time complexity is O(n).

As the number of records increases, the search time is expected to increase.

4.6 Complete the Code – PassStore

The PassStore program must complete the find() method using BufferedReader.

The method should:

- Read the file line by line.
- Skip damaged lines that do not contain exactly five fields.
- Compare the pass ID with the requested ID.
- Return the matching fields.
- Return null when the pass is not found.

Expected output:

[P1003, 192311003, Divya, 7, 3800.0]

null

The damaged P1002 record does not stop the search because the program skips lines that do not contain the required five fields.

4.7 Alternative Concept – HashMap

Instead of scanning the complete file for every search, all valid records can be loaded into a HashMap using pass ID as the key.

For n records and q searches:

- Repeated file scanning: O(q × n).
- Loading into a HashMap and searching: O(n + q) expected time.

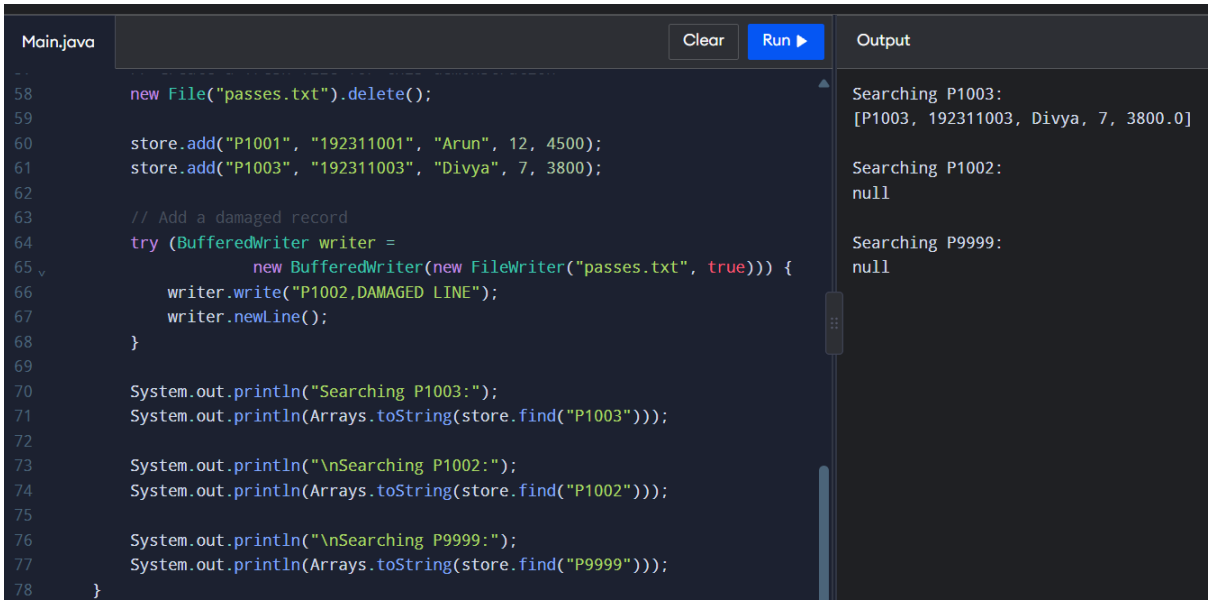
The HashMap requires additional memory of O(n), while repeated file scanning uses less additional memory.

A HashMap is useful when many searches are performed during the same application session.

PROGRAM:

```
import java.io.*;
import java.nio.file.*;
import java.util.*;
class PassStore {
   private String fileName = "passes.txt";
   public void add(String passId, String regNo, String name,
                   int route, double amount) throws IOException {
      try (BufferedWriter writer = new BufferedWriter(
            new FileWriter(fileName, true))) {
```

```
            writer.write(passId + "," + regNo + "," + name + ","
                    + route + "," + amount);
            writer.newLine();
        }
    }
    public String[] find(String passId) throws IOException {
        File file = new File(fileName);
        if (!file.exists()) {
            return null;
        }
        try (BufferedReader reader =
                new BufferedReader(new FileReader(fileName))) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] fields = line.split(",", -1);
                // Skip damaged lines
                if (fields.length != 5) {
                    continue;
                }
                if (fields[0].equals(passId)) {
                    return fields;
                }
            }
        }
        return null;
    }
}
public class Main {
    public static void main(String[] args) throws IOException {
        PassStore store = new PassStore();
        // Create a fresh file for this demonstration
        new File("passes.txt").delete();
        store.add("P1001", "192311001", "Arun", 12, 4500);
        store.add("P1003", "192311003", "Divya", 7, 3800);
        // Add a damaged record
        try (BufferedWriter writer =
                new BufferedWriter(new FileWriter("passes.txt", true))) {
            writer.write("P1002,DAMAGED LINE");
            writer.newLine();
        }
        System.out.println("Searching P1003:");
        System.out.println(Arrays.toString(store.find("P1003")));
        System.out.println("\nSearching P1002:");
        System.out.println(Arrays.toString(store.find("P1002")));
        System.out.println("\nSearching P9999:");
        System.out.println(Arrays.toString(store.find("P9999")));
    }
}
```



TASK B2 – ENGINEER AN ADVERSARIAL INPUT

4.8 Introduction

This task investigates how incorrect or deliberately crafted inputs can expose weaknesses in file handling and database authentication.

4.9 Unsafe and Safe Login Comparison

The unsafe login method uses string concatenation to construct SQL queries.

A crafted input may change the SQL condition and allow an invalid login to be accepted.

The safe method uses PreparedStatement, which treats user input as parameter values rather than SQL commands.

The experiment should demonstrate:

- The unsafe query accepting a deliberately crafted input.
- The PreparedStatement version rejecting the invalid login.
- The security difference between the two approaches.

4.10 Damaged Bus-Pass File Testing

The following damaged records should be tested:

1. A record containing the wrong number of fields.
2. A record containing a non-numeric amount.
3. An empty line.

The file store should validate each record and report or skip invalid records instead of terminating unexpectedly.

4.11 Alternative Concept – Input Escaping and Regular Expressions

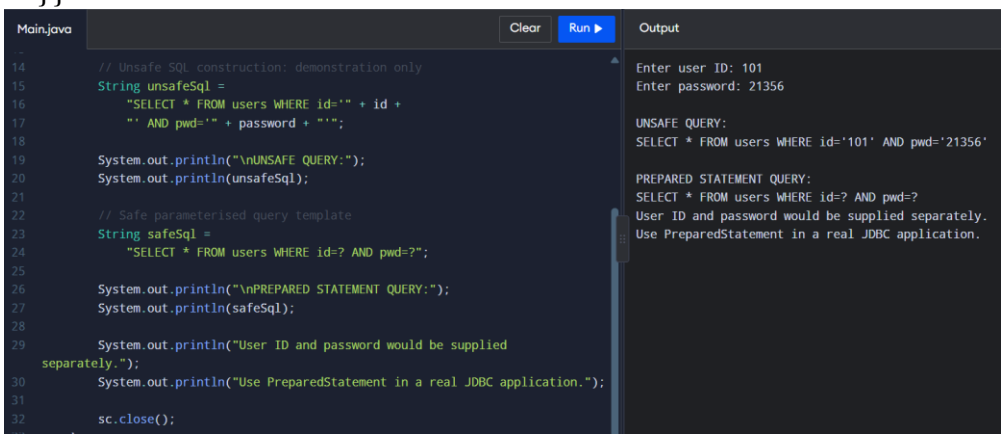
Escaping quote characters or validating input using regular expressions may reduce some risks, but these methods are weaker than PreparedStatement.

They require developers to correctly handle every possible input case and SQL context.

PreparedStatement separates SQL structure from user-supplied values, making it the preferred approach for parameterised queries.

PROGRAM:

```
import java.util.Scanner;
public class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter user ID: ");
      String id = sc.nextLine();
      System.out.print("Enter password: ");
      String password = sc.nextLine();
      // Unsafe SQL construction: demonstration only
      String unsafeSql =
         "SELECT * FROM users WHERE id='" + id +
         "' AND pwd='" + password + "'";
      System.out.println("\nUNSAFE QUERY:");
      System.out.println(unsafeSql);
      // Safe parameterised query template
      String safeSql =
         "SELECT * FROM users WHERE id=? AND pwd=?";
      System.out.println("\nPREPARED STATEMENT QUERY:");
      System.out.println(safeSql);
      System.out.println("User ID and password would be supplied separately.");
      System.out.println("Use PreparedStatement in a real JDBC application.");
      sc.close();
   }}
```



TASK B3 – STATEMENT VS PREPAREDSTATEMENT VS BATCH

4.12 Introduction

JDBC provides different methods for executing SQL statements.

This task compares Statement, PreparedStatement and PreparedStatement with batch processing when inserting bus-pass records into a MySQL database.

4.13 B3.1 – Correctness on Small Data

Ten bus-pass records must be inserted using each of the three approaches.

The records should be verified using a SELECT query and ResultSetMetaData.

The verification should confirm:

- All records are inserted correctly.
- The table contains the expected values.
- Column names are correct.
- Database column types are appropriate.

4.14 B3.2 – Increase the Data Size

The insertion performance must be measured for 1,000, 10,000 and 100,000 records.

Number of Records	Statement	PreparedStatement	Batch Processing
1,000	To be measured	To be measured	To be measured
10,000	To be measured	To be measured	To be measured
100,000	To be measured	To be measured	To be measured

Batch processing groups multiple insert operations and sends them together, reducing repeated database communication.

Actual timings must be collected through program execution.

4.15 B3.3 – Auto-Commit vs Transaction

Auto-commit makes each successful SQL statement permanent immediately.

A transaction allows several database operations to be treated as one unit.

If an error occurs halfway through the batch:

- With auto-commit enabled, previously committed rows may remain in the database.
- With a single transaction, rollback can undo the inserts performed within that transaction.

This demonstrates the importance of transaction management for data correctness.

4.16 B3.4 – Engineering Trade-Off

For a nightly import of 100,000 bus passes, PreparedStatement with batch processing and a transaction is a suitable approach.

For a single pass issued at a counter, PreparedStatement is appropriate because it safely inserts an individual record.

4.17 Complete the Code – PassLoader

The PassLoader program must complete three TODO sections.

TODO 1: Set all five SQL parameters, add each record to the batch and execute the batch after every 1,000 records.

TODO 2: Commit the transaction after all records are inserted and return the number of loaded records.

TODO 3: Roll back the transaction if an SQL or runtime exception occurs, then re-throw the exception.

The program must be tested with 1,000, 10,000 and 100,000 generated records.

An invalid amount should also be inserted halfway through the list to verify that the transaction rollback leaves no partially inserted records from that transaction.

4.18 Alternative Concept – Multi-Row INSERT and LOAD DATA

A multi-row INSERT can insert several records through a single SQL statement.

MySQL LOAD DATA can load large amounts of data from a file into a database.

These methods can reduce database communication overhead for large imports.

PreparedStatement batching provides parameterised insertion and allows the application to control batch size and transaction boundaries.

The actual performance should be compared using the measurements collected during execution.

PROGRAM:

```
import java.util.*;
public class Main {
   static ArrayList<String> database = new ArrayList<>();
   static int loadAll(ArrayList<String[]> passes) {
      ArrayList<String> temporary = new ArrayList<>();
      int batchSize = 1000;
      try {
         for (int i = 0; i < passes.size(); i++) {
            String[] p = passes.get(i);
            int route = Integer.parseInt(p[3]);
            double amount = Double.parseDouble(p[4]);
            if (amount <= 0) {
               throw new RuntimeException("Invalid amount at record "
                     + (i + 1));
            }
            temporary.add(
               p[0] + "," + p[1] + "," + p[2] + "," +
               route + "," + amount
            );
            if (temporary.size() % batchSize == 0) {
               System.out.println("Prepared batch of " +
                     temporary.size() + " records");
            }
         }
         // Commit: add all temporary records
         database.addAll(temporary);
         return temporary.size();
      } catch (RuntimeException e) {
         // Rollback: do not add temporary records
         System.out.println("Rollback completed.");
         System.out.println("Error: " + e.getMessage());
         return 0;
      }
   }
   public static void main(String[] args) {
      ArrayList<String[]> passes = new ArrayList<>();
      for (int i = 1; i <= 2500; i++) {
         passes.add(new String[] {
            "P" + i,
            "192311" + i,
            "Student" + i,
            "12",
            "4500"
         });
      }
      System.out.println("TEST 1: VALID BATCH INSERT");
      int inserted = loadAll(passes);
      System.out.println("Rows loaded: " + inserted);
      System.out.println("Database records: " + database.size());
```

```
        // Test failure with invalid amount
        ArrayList<String[]> invalidPasses = new ArrayList<>();
        for (int i = 1; i <= 10; i++) {
            String amount = (i == 5) ? "invalid" : "4500";
            invalidPasses.add(new String[] {
                "X" + i,
                "192322" + i,
                "Student" + i,
                "7",
                amount
            });
        }
        System.out.println("\nTEST 2: INVALID BATCH INSERT");
        int failedLoad = loadAll(invalidPasses);
        System.out.println("Rows loaded: " + failedLoad);
        System.out.println("Database records after failure: "
                + database.size());
    }
}
```



TASK B4 – FINAL ENGINEERING DECISION

4.19 Introduction

The transport office plans to expand its service to all 12 university campuses and handle 100 times the current number of bus passes.

Several counters may issue passes at the same time. Therefore, the storage system must support efficient searching, reliable updates and concurrent access.

4.20 B4.1 – Implement a Practical Strategy

A migration program should be designed to read existing bus-pass records from a text file and transfer valid records into the database.

The migration process should:

1. Open the existing bus-pass file.
2. Read records one by one.
3. Validate each record.
4. Identify invalid records.
5. Add valid records to JDBC batches.
6. Execute the batches within a transaction.
7. Commit successful database operations.
8. Roll back when a transaction fails.
9. Generate a report containing records that failed validation.

This approach helps maintain data correctness during migration.

4.21 B4.2 – Inspect the Database

JDBC DatabaseMetaData provides information about the connected database system and its driver.

The application should report:

- Database product name.
- Database product version.
- JDBC driver name or version.
- Tables present in the database.

This information is useful when moving the application to another server because it helps identify database and driver differences that may affect compatibility.

4.22 B4.3 – Production Decision

For the expanded transport office system, a relational database is the most suitable storage option.

Text Files:

Text files are simple to create and inspect, but searching and updating large records can become inefficient. They also require additional mechanisms for safe concurrent access.

Serialization:

Serialization is useful for storing Java object graphs, but it is less suitable for a large multi-campus system requiring shared access, structured queries and concurrent updates.

Database:

A relational database supports structured storage, SQL searching, transactions, constraints and concurrent access management.

Therefore, MySQL or another suitable relational database is recommended for the expanded bus-pass management system.

4.23 Alternative Concept – Separate Serialized Files for Each Campus

If each campus stores its own serialized file, the system must search multiple files to locate records across all campuses.

The search process may require checking all 12 files, increasing the overall search effort.

Each campus can manage its own local data, but sharing and synchronising records becomes more difficult.

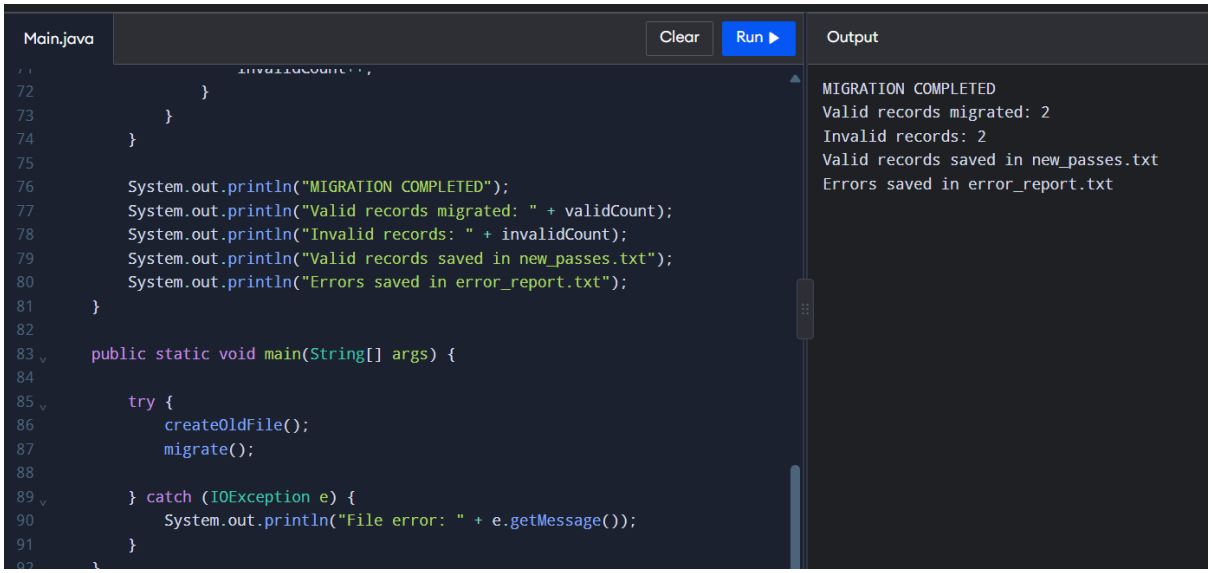
Concurrent counters must also be coordinated to prevent conflicting updates.

A shared database provides a centralised approach for searching, managing records and controlling concurrent updates.

PROGRAM:

```
import java.io.*;
public class Main {
   static void createOldFile() throws IOException {
      try (BufferedWriter writer =
                new BufferedWriter(new FileWriter("old_passes.txt"))) {
         writer.write("P1001,192311001,Arun,12,4500");
         writer.newLine();
         writer.write("P1002,192311002,Divya,7,3800");
         writer.newLine();
         writer.write("P1003,192311003,Kumar,ABC,4000");
         writer.newLine();
         writer.write("P1004,DAMAGED");
         writer.newLine();
      }
   }
   static void migrate() throws IOException {
      int validCount = 0;
      int invalidCount = 0;
      try (
         BufferedReader reader =
            new BufferedReader(new FileReader("old_passes.txt"));
         BufferedWriter validWriter =
            new BufferedWriter(new FileWriter("new_passes.txt"));
         BufferedWriter errorWriter =
```

```
            new BufferedWriter(new FileWriter("error_report.txt"))
        ) {
            String line;
            while ((line = reader.readLine()) != null) {
                try {
                    String[] fields = line.split(",", -1);
                    if (fields.length != 5) {
                        throw new Exception("Incorrect number of fields");
                    }
                    if (fields[0].isEmpty() || fields[1].isEmpty()
                            || fields[2].isEmpty()) {
                        throw new Exception("Missing required information");
                    }
                    Integer.parseInt(fields[3]);
                    double amount = Double.parseDouble(fields[4]);
                    if (amount <= 0) {
                        throw new Exception("Invalid amount");
                    }
                    validWriter.write(line);
                    validWriter.newLine();
                    validCount++;
                } catch (Exception e) {
                    errorWriter.write(line + " -> " + e.getMessage());
                    errorWriter.newLine();
                    invalidCount++;
                }
            }
        }
        System.out.println("MIGRATION COMPLETED");
        System.out.println("Valid records migrated: " + validCount);
        System.out.println("Invalid records: " + invalidCount);
        System.out.println("Valid records saved in new_passes.txt");
        System.out.println("Errors saved in error_report.txt");
    }
    public static void main(String[] args) {
        try {
            createOldFile();
            migrate();
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```



5. OVERALL RESULT

The assignment examines different Java persistence methods and their suitability for applications with increasing data requirements.

File handling provides a simple method for storing records, while buffering improves the efficiency of large-file processing. Serialization supports saving Java object graphs, and JDBC provides a connection between Java applications and relational databases.

The assignment also demonstrates the importance of transactions, PreparedStatement, batch processing and database metadata.

The final engineering decision is to use a relational database for the expanded university transport system because it provides better support for large-scale storage, searching, transaction safety and concurrent access.

6. CONCLUSION

This assignment provides an understanding of data persistence engineering using Java file handling, serialization and JDBC.The design tasks demonstrate how to select suitable storage formats, handle damaged data, improve file-reading performance and organise database access using the DAO pattern.The implementation tasks focus on testing file operations, measuring performance, identifying SQL injection vulnerabilities and comparing different database insertion methods.The study highlights that storing data is not the only requirement of a reliable application. Data must also be protected, validated, retrieved efficiently and managed correctly when multiple users access the system.For the university transport office expansion, a relational database with PreparedStatement, batch processing and transaction management is the recommended solution.